🛠️ Implementando Servicios

🎯 Objetivos de Aprendizaje

Al finalizar este tema, serás capaz de:

- ✅ Crear clases de Service correctamente
- ✅ Integrar Services con FastAPI usando Dependency Injection
- ✅ Manejar errores entre capas
- ✅ Estructurar proyectos con Service Layer

📚 Contenido

1. Anatomía de un Service

```python
# services/author_service.py
from sqlalchemy import select
from sqlalchemy.orm import Session

from models.author import Author
from schemas.author import AuthorCreate, AuthorUpdate


class AuthorService:
    """
    Service para operaciones de Author.

    Contiene toda la lógica de negocio relacionada con autores.
    """

    def __init__(self, db: Session):
        """
        Inicializa el service con una sesión de DB.

        Args:
            db: Sesión de SQLAlchemy
        """
        self.db = db

    def get_by_id(self, author_id: int) -> Author | None:
        """Obtiene un autor por ID"""
        return self.db.get(Author, author_id)

    def get_by_email(self, email: str) -> Author | None:
        """Obtiene un autor por email"""
        stmt = select(Author).where(Author.email == email)
        return self.db.execute(stmt).scalar_one_or_none()

    def list_all(self, skip: int = 0, limit: int = 10) -> list[Author]:
        """Lista autores con paginación"""
        stmt = select(Author).offset(skip).limit(limit)
        return self.db.execute(stmt).scalars().all()

    def create(self, author_data: AuthorCreate) -> Author:
```

```
        """
        Crea un nuevo autor.

        Raises:
            ValueError: Si el email ya existe
        """
        # Validación de negocio
        if self.get_by_email(author_data.email):
            raise ValueError(f"Email {author_data.email} already registered")

        # Crear autor
        author = Author(
            name=author_data.name,
            email=author_data.email,
            bio=author_data.bio
        )
        self.db.add(author)
        self.db.commit()
        self.db.refresh(author)

        return author

    def update(self, author_id: int, author_data: AuthorUpdate) -> Author:
        """
        Actualiza un autor existente.

        Raises:
            ValueError: Si el autor no existe
            ValueError: Si el nuevo email ya está en uso
        """
        author = self.get_by_id(author_id)
        if not author:
            raise ValueError(f"Author {author_id} not found")

        # Si actualiza email, verificar que no exista
        update_dict = author_data.model_dump(exclude_unset=True)
        if "email" in update_dict:
            existing = self.get_by_email(update_dict["email"])
            if existing and existing.id != author_id:
                raise ValueError(f"Email {update_dict['email']} already in use")

        # Aplicar cambios
        for key, value in update_dict.items():
            setattr(author, key, value)

        self.db.commit()
        self.db.refresh(author)

        return author

    def delete(self, author_id: int) -> None:
        """
        Elimina un autor.

        Raises:
            ValueError: Si el autor no existe
```

```
        """
        author = self.get_by_id(author_id)
        if not author:
            raise ValueError(f"Author {author_id} not found")

        self.db.delete(author)
        self.db.commit()
```

---

2. Integración con Router

```python
# routers/authors.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from database import get_db
from schemas.author import AuthorCreate, AuthorUpdate, AuthorResponse
from services.author_service import AuthorService

router = APIRouter(prefix="/authors", tags=["authors"])


@router.post("/", response_model=AuthorResponse, status_code=status.HTTP_201_CREATED)
def create_author(author: AuthorCreate, db: Session = Depends(get_db)):
    """Crea un nuevo autor"""
    service = AuthorService(db)
    try:
        return service.create(author)
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=str(e)
        )


@router.get("/", response_model=list[AuthorResponse])
def list_authors(
    skip: int = 0,
    limit: int = 10,
    db: Session = Depends(get_db)
):
    """Lista autores con paginación"""
    service = AuthorService(db)
    return service.list_all(skip=skip, limit=limit)


@router.get("/{author_id}", response_model=AuthorResponse)
def get_author(author_id: int, db: Session = Depends(get_db)):
    """Obtiene un autor por ID"""
    service = AuthorService(db)
    author = service.get_by_id(author_id)

    if not author:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Author {author_id} not found"
```

```
        )

    return author


@router.put("/{author_id}", response_model=AuthorResponse)
def update_author(
    author_id: int,
    author_data: AuthorUpdate,
    db: Session = Depends(get_db)
):
    """Actualiza un autor"""
    service = AuthorService(db)
    try:
        return service.update(author_id, author_data)
    except ValueError as e:
        # Determinar código HTTP según el error
        if "not found" in str(e).lower():
            raise HTTPException(status_code=404, detail=str(e))
        raise HTTPException(status_code=409, detail=str(e))


@router.delete("/{author_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_author(author_id: int, db: Session = Depends(get_db)):
    """Elimina un autor"""
    service = AuthorService(db)
    try:
        service.delete(author_id)
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e))
```

---

3. Service con Relaciones

```
# services/post_service.py
from sqlalchemy import select, desc
from sqlalchemy.orm import Session, joinedload, selectinload

from models.post import Post
from models.author import Author
from models.tag import Tag
from schemas.post import PostCreate, PostUpdate


class PostService:
    """Service para operaciones de Post"""

    def __init__(self, db: Session):
        self.db = db

    def get_by_id(self, post_id: int) -> Post | None:
        """Obtiene un post con sus relaciones"""
        stmt = (
            select(Post)
            .options(
                joinedload(Post.author),
```

```
            selectinload(Post.tags)
        )
        .where(Post.id == post_id)
    )
    return self.db.execute(stmt).scalar_one_or_none()

def list_all(
    self,
    skip: int = 0,
    limit: int = 10,
    author_id: int | None = None,
    tag_name: str | None = None
) -> list[Post]:
    """Lista posts con filtros opcionales"""
    stmt = (
        select(Post)
        .options(
            joinedload(Post.author),
            selectinload(Post.tags)
        )
    )

    # Filtrar por autor
    if author_id:
        stmt = stmt.where(Post.author_id == author_id)

    # Filtrar por tag
    if tag_name:
        stmt = stmt.join(Post.tags).where(Tag.name == tag_name)

    stmt = stmt.order_by(desc(Post.created_at)).offset(skip).limit(limit)

    return self.db.execute(stmt).scalars().unique().all()

def create(self, post_data: PostCreate) -> Post:
    """
    Crea un nuevo post.

    Raises:
        ValueError: Si el autor no existe
    """
    # Validar que el autor existe
    author = self.db.get(Author, post_data.author_id)
    if not author:
        raise ValueError(f"Author {post_data.author_id} not found")

    # Obtener o crear tags
    tags = self._get_or_create_tags(post_data.tag_names or [])

    # Crear post
    post = Post(
        title=post_data.title,
        content=post_data.content,
        author_id=post_data.author_id,
        tags=tags
    )
```

```
        self.db.add(post)
        self.db.commit()
        self.db.refresh(post)

        return post

    def update(self, post_id: int, post_data: PostUpdate) -> Post:
        """Actualiza un post existente"""
        post = self.db.get(Post, post_id)
        if not post:
            raise ValueError(f"Post {post_id} not found")

        update_dict = post_data.model_dump(exclude_unset=True)

        # Si actualiza tags
        if "tag_names" in update_dict:
            post.tags = self._get_or_create_tags(update_dict.pop("tag_names"))

        # Si actualiza author_id, validar
        if "author_id" in update_dict:
            author = self.db.get(Author, update_dict["author_id"])
            if not author:
                raise ValueError(f"Author {update_dict['author_id']} not found")

        # Aplicar otros cambios
        for key, value in update_dict.items():
            setattr(post, key, value)

        self.db.commit()
        self.db.refresh(post)

        return post

    def delete(self, post_id: int) -> None:
        """Elimina un post"""
        post = self.db.get(Post, post_id)
        if not post:
            raise ValueError(f"Post {post_id} not found")

        self.db.delete(post)
        self.db.commit()

    def add_tag(self, post_id: int, tag_name: str) -> Post:
        """Agrega un tag a un post"""
        post = self.db.get(Post, post_id)
        if not post:
            raise ValueError(f"Post {post_id} not found")

        tag = self._get_or_create_tag(tag_name)

        if tag not in post.tags:
            post.tags.append(tag)
            self.db.commit()

        return post
```

```python
    def remove_tag(self, post_id: int, tag_name: str) -> Post:
        """Elimina un tag de un post"""
        post = self.db.get(Post, post_id)
        if not post:
            raise ValueError(f"Post {post_id} not found")

        for tag in post.tags:
            if tag.name == tag_name:
                post.tags.remove(tag)
                self.db.commit()
                break

        return post

    # Métodos privados
    def _get_or_create_tag(self, name: str) -> Tag:
        """Obtiene un tag o lo crea si no existe"""
        stmt = select(Tag).where(Tag.name == name)
        tag = self.db.execute(stmt).scalar_one_or_none()

        if not tag:
            tag = Tag(name=name)
            self.db.add(tag)
            self.db.flush()  # Para obtener el ID sin commit

        return tag

    def _get_or_create_tags(self, names: list[str]) -> list[Tag]:
        """Obtiene o crea múltiples tags"""
        return [self._get_or_create_tag(name) for name in names]
```

---

4. Excepciones Personalizadas (Opcional)

Para mejor organización de errores:

```python
# exceptions.py
class ServiceError(Exception):
    """Base exception for service errors"""
    pass

class NotFoundError(ServiceError):
    """Resource not found"""
    pass

class DuplicateError(ServiceError):
    """Duplicate resource"""
    pass

class ValidationError(ServiceError):
    """Business validation failed"""
    pass


# services/author_service.py
from exceptions import NotFoundError, DuplicateError
```

```python
class AuthorService:
    def create(self, author_data: AuthorCreate) -> Author:
        if self.get_by_email(author_data.email):
            raise DuplicateError(f"Email {author_data.email} already registered")
        # ...


# routers/authors.py
from exceptions import NotFoundError, DuplicateError

@router.post("/", response_model=AuthorResponse)
def create_author(author: AuthorCreate, db: Session = Depends(get_db)):
    service = AuthorService(db)
    try:
        return service.create(author)
    except DuplicateError as e:
        raise HTTPException(status_code=409, detail=str(e))
    except NotFoundError as e:
        raise HTTPException(status_code=404, detail=str(e))
```

---

5. Configurando main.py

```python
# main.py
from fastapi import FastAPI
from database import engine, Base
from routers import authors, posts

# Crear tablas
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="Blog API",
    description="API con Service Layer",
    version="1.0.0"
)

# Registrar routers
app.include_router(authors.router)
app.include_router(posts.router)


@app.get("/health")
def health_check():
    return {"status": "healthy"}
```

---

6. Testing de Services

Una gran ventaja es poder testear sin HTTP:

```python
# tests/test_author_service.py
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
```

```python
from database import Base
from services.author_service import AuthorService
from schemas.author import AuthorCreate

# Setup de test
engine = create_engine("sqlite:///:memory:")
TestSession = sessionmaker(bind=engine)

@pytest.fixture
def db():
    Base.metadata.create_all(bind=engine)
    session = TestSession()
    yield session
    session.close()
    Base.metadata.drop_all(bind=engine)

def test_create_author(db):
    service = AuthorService(db)

    author_data = AuthorCreate(
        name="John Doe",
        email="john@example.com"
    )

    author = service.create(author_data)

    assert author.id is not None
    assert author.name == "John Doe"
    assert author.email == "john@example.com"

def test_create_duplicate_email_raises(db):
    service = AuthorService(db)

    author_data = AuthorCreate(name="John", email="john@example.com")
    service.create(author_data)

    # Intentar crear otro con mismo email
    with pytest.raises(ValueError, match="already registered"):
        service.create(author_data)
```

✅ Checklist

- [ ] Sé crear clases Service con métodos CRUD
- [ ] Puedo integrar Services con routers
- [ ] Manejo errores entre Service y Router
- [ ] Puedo testear Services independientemente

← Anterior: Introducción Service Layer | Volver a README →